

การสร้าง Singly Circular Linked List

เมื่อเริ่มรันโปรแกรม จะสร้าง Head Node ขึ้นมาก่อน โดยกำหนดค่า

```
#define HeadData -999
```

จากนั้นสร้างข้อมูลแบบสุ่มจำนวน 10 โหนด และเชื่อมโยงกันเป็นลิ่งค์ลิสต์เดี่ยววงกลม (Singly Circular Linked List) โดยโหนดสุดท้ายจะชี้กลับไปที่ Head Node เสมอ ดังนี้

```
PROGRAM SINGLY CIRCULAR LINKED LIST
=====
All Data in Linked List
H = 508d69d0
1) : 508d69f0 INFO : 42 LINK : 508d6a10
2) : 508d6a10 INFO : 54 LINK : 508d6a30
3) : 508d6a30 INFO : 98 LINK : 508d6a50
4) : 508d6a50 INFO : 68 LINK : 508d6a70
5) : 508d6a70 INFO : 63 LINK : 508d6a90
6) : 508d6a90 INFO : 83 LINK : 508d6ab0
7) : 508d6ab0 INFO : 94 LINK : 508d6ad0
8) : 508d6ad0 INFO : 55 LINK : 508d6af0
9) : 508d6af0 INFO : 35 LINK : 508d6b10
10) : 508d6b10 INFO : 12 LINK : 508d69d0
MENU : [I:Insert D:Delete E:Exit]:
```

จะเห็นว่า Node สุดท้ายมี LINK ชี้กลับไปยัง H จึงเป็น Circular Linked List อย่างถูกต้อง
การแทรกข้อมูล

```
MENU : [I:Insert D:Delete E:Exit]: i

Insert After data : 54

Insert data : 100

All Data in Linked List AFTER INSERTED
H = 508d69d0
1) : 508d69f0 INFO : 42 LINK : 508d6a10
2) : 508d6a10 INFO : 54 LINK : 508d1260
3) : 508d1260 INFO : 100 LINK : 508d6a30
4) : 508d6a30 INFO : 98 LINK : 508d6a50
5) : 508d6a50 INFO : 68 LINK : 508d6a70
6) : 508d6a70 INFO : 63 LINK : 508d6a90
7) : 508d6a90 INFO : 83 LINK : 508d6ab0
8) : 508d6ab0 INFO : 94 LINK : 508d6ad0
9) : 508d6ad0 INFO : 55 LINK : 508d6af0
10) : 508d6af0 INFO : 35 LINK : 508d6b10
11) : 508d6b10 INFO : 12 LINK : 508d69d0
MENU : [I:Insert D:Delete E:Exit]:
```

โปรแกรมทำการค้นหา Node ที่มีค่า 54 จากนั้นสร้าง Node ใหม่เก็บค่า 100 และเชื่อม Pointer ตามหลักการ $LINK(P)=LINK(H1)LINK(H1)=P$ ทำให้ Node ใหม่ถูกแทรกหลัง Node ค่า 54 สำเร็จ

การแทรกหลัง Node สุดท้าย

```
MENU : [I:Insert D:Delete E:Exit]: i

Insert After data : 12

Insert data : 200

All Data in Linked List AFTER INSERTED
H = 508d69d0
1) : 508d69f0  INFO : 42      LINK : 508d6a10
2) : 508d6a10  INFO : 54      LINK : 508d1260
3) : 508d1260  INFO : 100     LINK : 508d6a30
4) : 508d6a30  INFO : 98      LINK : 508d6a50
5) : 508d6a50  INFO : 68      LINK : 508d6a70
6) : 508d6a70  INFO : 63      LINK : 508d6a90
7) : 508d6a90  INFO : 83      LINK : 508d6ab0
8) : 508d6ab0  INFO : 94      LINK : 508d6ad0
9) : 508d6ad0  INFO : 55      LINK : 508d6af0
10) : 508d6af0  INFO : 35      LINK : 508d6b10
11) : 508d6b10  INFO : 12      LINK : 508d1280
12) : 508d1280  INFO : 200     LINK : 508d69d0
MENU : [I:Insert D:Delete E:Exit]:
```

Node ใหม่ถูกสร้างต่อท้ายลิสต์ และ LINK ของ Node ใหม่ชี้กลับไปยัง Head Node ทำให้ยังคงเป็น Circular Linked List อยู่

การลบข้อมูล

```
MENU : [I:Insert D:Delete E:Exit]: d

Delete After data : 100

All Data in Linked List AFTER DELETED
H = 508d69d0
1) : 508d69f0  INFO : 42      LINK : 508d6a10
2) : 508d6a10  INFO : 54      LINK : 508d1260
3) : 508d1260  INFO : 100     LINK : 508d6a50
4) : 508d6a50  INFO : 68      LINK : 508d6a70
5) : 508d6a70  INFO : 63      LINK : 508d6a90
6) : 508d6a90  INFO : 83      LINK : 508d6ab0
7) : 508d6ab0  INFO : 94      LINK : 508d6ad0
8) : 508d6ad0  INFO : 55      LINK : 508d6af0
9) : 508d6af0  INFO : 35      LINK : 508d6b10
10) : 508d6b10  INFO : 12      LINK : 508d1280
11) : 508d1280  INFO : 200     LINK : 508d69d0
MENU : [I:Insert D:Delete E:Exit]:
```

โปรแกรมจะลบ Node ที่อยู่ถัดจาก Node ค่า 100 และปรับ Pointer ใหม่ให้เชื่อมต่อกันอย่างถูกต้อง

การลบ Node สุดท้าย

```
MENU : [I:Insert D:Delete E:Exit]: d

Delete After data : 12

All Data in Linked List AFTER DELETED
H = 508d69d0
1) : 508d69f0 INFO : 42 LINK : 508d6a10
2) : 508d6a10 INFO : 54 LINK : 508d1260
3) : 508d1260 INFO : 100 LINK : 508d6a50
4) : 508d6a50 INFO : 68 LINK : 508d6a70
5) : 508d6a70 INFO : 63 LINK : 508d6a90
6) : 508d6a90 INFO : 83 LINK : 508d6ab0
7) : 508d6ab0 INFO : 94 LINK : 508d6ad0
8) : 508d6ad0 INFO : 55 LINK : 508d6af0
9) : 508d6af0 INFO : 35 LINK : 508d6b10
10) : 508d6b10 INFO : 12 LINK : 508d69d0
MENU : [I:Insert D:Delete E:Exit]:
```

ระบบจะเชื่อม LINK ของ Node 12 ให้กลับมายัง Head Node โดยตรง และคืนหน่วยความจำของ Node ที่ถูกลบ

การป้องกันการลบ Head Node

```
MENU : [I:Insert D:Delete E:Exit]: d

Delete After data : 12
This is the HEAD Node,Can't delete it!!!

All Data in Linked List AFTER DELETED
H = 508d69d0
1) : 508d69f0 INFO : 42 LINK : 508d6a10
2) : 508d6a10 INFO : 54 LINK : 508d1260
3) : 508d1260 INFO : 100 LINK : 508d6a50
4) : 508d6a50 INFO : 68 LINK : 508d6a70
5) : 508d6a70 INFO : 63 LINK : 508d6a90
6) : 508d6a90 INFO : 83 LINK : 508d6ab0
7) : 508d6ab0 INFO : 94 LINK : 508d6ad0
8) : 508d6ad0 INFO : 55 LINK : 508d6af0
9) : 508d6af0 INFO : 35 LINK : 508d6b10
10) : 508d6b10 INFO : 12 LINK : 508d69d0
MENU : [I:Insert D:Delete E:Exit]:
```

เนื่องจาก Node ถัดจาก 35 คือ Head Node โปรแกรมจึงไม่อนุญาตให้ลบตามคุณสมบัติของ Circular Linked List ที่กำหนดไว้ในเอกสาร

การประกาศโครงสร้าง Node

```
typedef struct Node    // Declare structure of node
{
    int info;
    struct Node *link;
} Node;
```

ใช้เก็บข้อมูลของแต่ละโหนด ประกอบด้วย info เก็บข้อมูล link เก็บตำแหน่งของโหนดถัดไป

```
Node *Allocate() // Allocate 1 node from storage pool
{
    struct Node *temp;
    temp = (Node *)malloc(sizeof(Node)); // Allocate node by
size declare
    return (temp);
}
```

ใช้จองพื้นที่หน่วยความจำสำหรับ Node ใหม่ โดยใช้ฟังก์ชัน malloc() และคืนค่าที่อยู่ของ Node กลับไปใช้งาน

```
void CreateNNode(int n) // Create N Node put data and link it
{
    int i, temp;
    H = p;
    H1 = p;
    for (i = 1; i <= n; i++) // Count N Node
    {
        p = Allocate();           // Allocate Node
        temp = 1 + rand() % 99; // random difference number
1..99
        p->info = temp;           // Put random data in to node
        H1->link = p;             // Let Last node point to new
node
        H1 = p;                  // Let H1 point to new node
        H1->link = H;             // Set link of H1 to HEAD for
Circular Linked List
    }
}
```

ใช้สร้าง Node จำนวน n โหนด ขั้นตอนการทำงาน สร้าง Node ใหม่ สุ่มข้อมูล 1-99
เก็บข้อมูลลงใน Node เชื่อมต่อกับ Node ก่อนหน้า ให้ Node สุดท้ายชี้กลับมายัง Head Node
ผลลัพธ์คือได้ Singly Circular Linked List

```
void ShowAllNode()
{
    printf("H = %x\n", H);          // Display address of pointer
H
    p = H->link;                    // Set start point of p
pointer at first node
    i = 1;                          // set start value of counter
    while (p->info != HeadData) // While if INFO is NOT
HeadData
    {
        printf("%d) : %x\t", i, p);    // Show COUNTER and
POINTER
        printf("INFO : %d\t", p->info); // Show INFO
        printf("LINK : %x\n", p->link); // Show LINK
        p = p->link;                    // Skip to next node
        i++;                            // Skip Counter
    } // End While
} // Enf Fn.
```

ใช้แสดงข้อมูลทุก Node ภายในลิสต์ ข้อมูลที่แสดงได้แก่ Address ของ Node ค่า INFO ค่า LINK
โดยเริ่มจาก H->link และวนไปเรื่อย ๆ จนกลับมาถึง Head Node

```
void InsertAfter(int data1)
{
    int temp;          // Temporary variable
    if (H->link == H) // If Link point back to HeadNode
        printf("Circular Linked List have no node!!..\n");
    else
    {
        H1 = H->link;    // Let H1 point at first
node
        while (H1->info != HeadData) // Search for the data
while INFO<> HeadData
        {
            if (H1->info == data1) // if Found
            {
```



```

        p = Allocate();                // Allocat one
node from storage pool
        printf("\nInsert data : "); // Input data for
insert
        scanf("%d", &temp);           // Read from KBD
        p->info = temp;                // Entry temporary
data into INFO of node
        p->link = H1->link;            // Change pointer
1st for insert node(FAR)
        H1->link = p;                  // Change pointer
2nd for insert node (NEAR)
    } // End if
    H1 = H1->link; // Skip H1 to next node
} // End while
} // End IF
} // End Fn.

```

ใช้แทรก Node ใหม่หลังข้อมูลที่ใช้ระบุ ขั้นตอน ค้นหาข้อมูลที่ต้องการ สร้าง Node ใหม่
รับค่าข้อมูลจากผู้ใช้ เปลี่ยน Pointer p->link = H1->link; H1->link = p; Node
ใหม่ถูกแทรกสำเร็จ

```

void DeleteAfter(int data1)
{
    int temp;           // Temporary variable
    if (H->link == H) // If Link point back to HeadNode
        printf("Circular Linked List have no node!!..\n");
    else
    {
        H1 = H->link;           // Let H1 point at start
node
        while (H1->info != HeadData) // Search for the data
while INFO<> HeadData
        {
            if (H1->info == data1) // if Found
            {
                if (H1->link == H) // If no more node
                    printf("This is the HEAD Node,Can't delete
it!!!\n");
                else
                {

```

```

        p = H1->link;        // Mark at node for
Delete
        if (p->link == H) // If p is last node
            H1->link = H; // Set link of H1 to
Head node
        else
            H1->link = p->link; // If not set link
of H1 point same address of p
            free(p);           // Free node to
storage pool
    } // End if2
} // End if1
H1 = H1->link; // Skip H1 to next node
} // End while
} // End IF
} // End Fn.

```

ใช้ลบ Node ถัดจากข้อมูลที่ใช้ระบุ ขั้นตอน ค้นหา Node เป้าหมาย ตรวจสอบว่า Node ถัดไปเป็น Head หรือไม่ ถ้าไม่ใช่ Head ให้ลบ Node ปรับ Pointer ใหม่ คืนหน่วยความจำด้วย free(p);

```

int main() // MAIN Fn.
{
    p = Allocate();        // Create HEAD Node
    p->info = HeadData;    // Assign Special data
    p->link = p;           // Link back to Node
    n = 10;                // Set amount of node
    CreateNNode(n);        // Call Fn. Create N nodes
    printf("PROGRAM SINGLY CIRCULAR LINKED LIST \n");
    printf("===== \n");
    printf("All Data in Linked List \n");
    ShowAllNode(); // Call Fn. Show all node
    ch = ' ';
    while (ch != 'E' && ch != 'e')
    {
        printf("MENU : [I:Insert D:Delete E:Exit]: ");
        scanf(" %c", &ch); // GCC/MinGW not working for
conio.h -> getch()
        switch (ch)
        {
            case 'i':
            case 'I':

```

```

        printf("\nInsert After data : "); // Input data
for insert after
        scanf("%d", &data);
        InsertAfter(data); // Call Fn. Insert after data
        printf("\nAll Data in Linked List AFTER
INSERTED\n");
        ShowAllNode(); // Call Fn. Show all node
        break;
    case 'd':
    case 'D':
        printf("\nDelete After data : "); // Input data
for Delete after
        scanf("%d", &data);
        DeleteAfter(data); // Call Fn. Delete after data
        printf("\nAll Data in Linked List AFTER
DELETED\n");
        ShowAllNode(); // Call Fn. Show all node
        break;
    } // End Switch...case
} // End While
return (0);
} // End MAIN

```

ทำหน้าที่ควบคุมการทำงานของโปรแกรม ประกอบด้วย สร้าง Head Node สร้าง Circular Linked List แสดงข้อมูลทั้งหมด รับคำสั่งจากผู้ใช้ I = Insert, D = Delete, E = Exit

สรุปผลการทดลอง

จากการทดลองสร้างโปรแกรม Singly Circular Linked List พบว่าสามารถสร้างลิสต์แบบวงกลมที่มี Head Node ได้สำเร็จ โดยโหนดสุดท้ายชี้กลับมายัง Head Node ตลอดเวลา

ทำให้สามารถเดินทางภายในลิสต์ได้อย่างต่อเนื่องโดยไม่เกิดค่า NULL

นอกจากนี้โปรแกรมสามารถเพิ่มข้อมูล (Insert) และลบข้อมูล (Delete)

ตามตำแหน่งที่ต้องการได้อย่างถูกต้อง รวมทั้งสามารถป้องกันการลบ Head Node

ตามเงื่อนไขของโครงสร้างข้อมูลแบบ Singly Circular Linked List ได้

ผลการทดสอบแสดงให้เห็นว่าโปรแกรมทำงานสอดคล้องกับหลักการและอัลกอริทึมที่ระบุไว้ในเอกสารการเรียนการสอนทุกประการ